

Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Claude and Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)

Searched Gemini on google, used the Claude application

c. Prompt used. (You must use the same prompt for both GenAIs.)

Uploaded the assignment instructions and rubric and typed "Write one of the programs"

d. Generated code from both GenAIs.

Claude:

```
/*
 *
 *=====
 *
 * Program Name:    EECS 348 Assignment 3 - CEO Email Priority Queue
 *
 * Description:    C++ object-oriented program that prioritizes emails for
a
 *                busy company CEO. Emails are stored in a MaxHeap that
is
 *                built from scratch on top of a list (std::vector) and
used
 *                as a priority queue. Priority is decided first by
sender
 *                category (Boss > Subordinate > Peer > ImportantPerson >
 *                OtherPerson) and then by date, so the newest email from
a
 *                category is read first.
 *
 * Inputs:        A test file of commands (EMAIL, NEXT, READ, COUNT). The
 *                file name is passed as the first command-line argument;
if
 *                it is missing, the user is prompted to type it.
 *
 * Output:        Terminal output for the NEXT and COUNT commands (and an
 *                error message if the file cannot be opened).
 *
 * Collaborators:  None
 *
 * Other Sources:  Code generated by Claude (Anthropic GenAI), accessed
through
 *                the Claude desktop app, from the Assignment 3 prompt.
```

```

*
* Author:          Erick Marte
* Creation Date:   10-01-2026
* Revision Date:   10-01-2026
* Revisions:       Initial version generated by Claude.
*
* Build:           g++ -std=c++11 -Wall -o assignment3
claude_email_priority.cpp
* Run:             ./assignment3 test.txt
*
=====
==
*/

#include <iostream>    // std::cout, std::cin, std::cerr for terminal I/O
#include <fstream>      // std::ifstream for reading the test file
#include <sstream>      // std::istringstream for parsing the date string
#include <string>       // std::string for text fields
#include <vector>       // std::vector is the list that backs the MaxHeap
#include <cstdint>      // std::size_t for heap indexes
#include <cctype>       // std::isspace for trimming whitespace
#include <utility>      // std::swap for exchanging heap elements

/*
-----
--
* Email class (source: Claude)
* Stores one email and knows how to compare its priority with another
email
* and how to display itself in the required NEXT format.
*
-----
*/

class Email {
public:
    // Default constructor so Email can be stored in a std::vector
    Email() : rank_(0), dateKey_(0), sequence_(0) {}

    // Build an email from its three text fields plus its arrival order
    Email(const std::string& sender, const std::string& subject,

```

```

        const std::string& date, long sequence)
    : sender_(sender),           // save sender category text
      subject_(subject),       // save subject line text
      date_(date),             // save date text exactly as
given
      rank_(categoryRank(sender)), // convert category to a
numeric rank
      dateKey_(dateToKey(date)), // convert MM-DD-YYYY to
YYYYMMDD
      sequence_(sequence) {}    // save arrival order for
tie-breaks

    // operator< means "this email has LOWER priority than other"
    // The MaxHeap uses it to keep the highest-priority email at the root
    bool operator<(const Email& other) const {
        if (rank_ != other.rank_)           // different sender
categories?
            return rank_ < other.rank_;     // lower rank = lower
priority
        if (dateKey_ != other.dateKey_)     // same category, different
dates?
            return dateKey_ < other.dateKey_; // older date = lower
priority
        return sequence_ < other.sequence_; // same date: earlier
arrival is lower
    }

    // Print this email in the NEXT command format
    void display(std::ostream& out) const {
        out << "Next email:\n";           // header line
        out << "\tSender: " << sender_ << "\n"; // tab-indented sender
        out << "\tSubject: " << subject_ << "\n"; // tab-indented subject
        out << "\tDate: " << date_ << "\n"; // tab-indented date
    }

private:
    std::string sender_; // sender category, e.g. "Boss"
    std::string subject_; // subject line (may contain spaces)
    std::string date_; // original date text, e.g. "01-03-2025"

```

```

    int rank_;           // numeric category priority (5 = Boss ... 1 =
OtherPerson)

    long dateKey_;       // date as YYYYMMDD so newer dates are larger
numbers

    long sequence_;      // order the email was added (later = larger)

// Map a sender category to its priority rank (higher = read sooner)
static int categoryRank(const std::string& sender) {
    if (sender == "Boss") return 5;           // read first
    if (sender == "Subordinate") return 4;    // read second
    if (sender == "Peer") return 3;           // read third
    if (sender == "ImportantPerson") return 2; // read fourth
    if (sender == "OtherPerson") return 1;    // read last
    return 0;                                 // unknown category:
lowest
}

// Convert "MM-DD-YYYY" into the number YYYYMMDD for easy comparison
static long dateToKey(const std::string& date) {
    std::istringstream in(date); // stream over the date text
    int month = 0;                // month part
    int day = 0;                  // day part
    int year = 0;                 // year part
    char dash1 = 0;               // first '-' separator
    char dash2 = 0;               // second '-' separator
    if (in >> month >> dash1 >> day >> dash2 >> year) //
parse all parts
        return static_cast<long>(year) * 10000L + month * 100L + day;
// YYYYMMDD
    return 0;                     // malformed date sorts as oldest
}
};

/*
-----
--
* MaxHeap class template (source: Claude)
* A list-based binary max heap built from scratch on a std::vector.
* For the element at index i:
*     parent = (i - 1) / 2, left child = 2i + 1, right child = 2i + 2

```

```

* T must provide operator< meaning "lower priority than".
*
-----
*/
template <typename T>
class MaxHeap {
public:
    // Add an item to the end of the list, then move it up to its spot
    void insert(const T& item) {
        items_.push_back(item);           // place new item at the next
open slot
        siftUp(items_.size() - 1);        // restore heap order upward
    }

    // Return the highest-priority item (caller must check isEmpty()
first)
    const T& top() const {
        return items_.front();           // the root holds the maximum
    }

    // Remove the highest-priority item; does nothing if the heap is empty
    void removeTop() {
        if (items_.empty()) return;       // nothing to remove
        items_.front() = items_.back();   // move last item into the root
        items_.pop_back();                // drop the now-duplicate last
slot
        if (!items_.empty())              // if anything is left...
            siftDown(0);                  // ...restore heap order downward
    }

    // Number of items currently stored
    std::size_t size() const {
        return items_.size();            // vector size is the heap size
    }

    // True when the heap has no items
    bool isEmpty() const {
        return items_.empty();            // vector empty means heap empty
    }

```

```

private:
    std::vector<T> items_; // the list that stores the heap level by
level

    // Index of the parent of node i
    static std::size_t parent(std::size_t i) { return (i - 1) / 2; }

    // Index of the left child of node i
    static std::size_t leftChild(std::size_t i) { return 2 * i + 1; }

    // Index of the right child of node i
    static std::size_t rightChild(std::size_t i) { return 2 * i + 2; }

    // Move the item at index i up while it outranks its parent
    void siftUp(std::size_t i) {
        while (i > 0 && items_[parent(i)] < items_[i]) { // parent is
lower priority
            std::swap(items_[i], items_[parent(i)]); // swap child
and parent
            i = parent(i); // continue from
parent's spot
        }
    }

    // Move the item at index i down while a child outranks it
    void siftDown(std::size_t i) {
        const std::size_t n = items_.size(); // number of
items in heap
        while (true) { // loop until
settled
            std::size_t largest = i; // assume
current is largest
            const std::size_t left = leftChild(i); // left child
index
            const std::size_t right = rightChild(i); // right child
index
            if (left < n && items_[largest] < items_[left]) // left
child is higher?
                largest = left; // remember
left

```

```

        if (right < n && items_[largest] < items_[right]) // right
child is higher?
            largest = right; // remember
right
        if (largest == i) break; // heap order
restored
        std::swap(items_[i], items_[largest]); // swap with
larger child
        i = largest; // continue
from child's spot
    }
}
};

/*
-----
--
* CEOInbox class (source: Claude)
* The CEO's priority queue of unread emails. Wraps a MaxHeap<Email> and
* provides one method per test-file command.
*
-----
*/

class CEOInbox {
public:
    // Start with an empty inbox and arrival counter at zero
    CEOInbox() : nextSequence_(0) {}

    // EMAIL command: add a new email to the priority queue
    void addEmail(const std::string& sender, const std::string& subject,
                  const std::string& date) {
        heap_.insert(Email(sender, subject, date, nextSequence_++)); //
insert and bump counter
    }

    // NEXT command: show the highest-priority email without removing it
    void showNext(std::ostream& out) const {
        if (heap_.isEmpty()) { // no emails left?
            out << "No emails to read.\n\n"; // tell the user, then
blank line
        }
    }
};

```

```

        return; // nothing else to show
    }
    heap_.top().display(out); // print the top email
    out << "\n"; // blank line after the
block
    }

    // READ command: the CEO dealt with the top email, so remove it
    void readEmail() {
        heap_.removeTop(); // safe even when the heap
is empty
    }

    // COUNT command: show how many unread emails remain
    void showCount(std::ostream& out) const {
        out << "There are " << heap_.size() // current number of emails
            << " emails to read.\n\n"; // message plus blank line
    }

private:
    MaxHeap<Email> heap_; // priority queue of unread emails
    long nextSequence_; // arrival order assigned to the next email
};

/*
-----
--
* CommandProcessor class (source: Claude)
* Reads the test file line by line, parses each command, and calls the
* matching CEOInbox method.
*
-----
*/

class CommandProcessor {
public:
    // Keep a reference to the inbox that commands will act on
    explicit CommandProcessor(CEOInbox& inbox) : inbox_(inbox) {}

    // Open the file and process every line; returns false if it cannot
open

```

```

    bool processFile(const std::string& fileName) {
        std::ifstream file(fileName.c_str()); // open the test file
        if (!file) {                          // did the open fail?
            std::cerr << "Error: could not open file \"" << fileName <<
"\n"; // report it
            return false;                      // signal failure to caller
        }
        std::string line;                     // holds one line of the
file
        while (std::getline(file, line))      // read until end of file
            processLine(line);                // handle each line
        return true;                          // file processed
successfully
    }

private:
    CEOInbox& inbox_; // the inbox that receives the commands

    // Remove leading/trailing whitespace (also strips Windows '\r')
    static std::string trim(const std::string& text) {
        std::size_t start = 0;                // first non-space
index
        std::size_t end = text.size();        // one past last
non-space
        while (start < end && std::isspace(static_cast<unsigned
char>(text[start])))
            ++start;                          // skip leading
spaces
        while (end > start && std::isspace(static_cast<unsigned
char>(text[end - 1])))
            --end;                            // skip trailing
spaces / '\r'
        return text.substr(start, end - start); // return the
trimmed part
    }

    // Decide which command a line holds and run it
    void processLine(const std::string& rawLine) {
        const std::string line = trim(rawLine); // clean up the
line

```

```

        if (line.empty()) return; // skip blank lines
        if (line.compare(0, 5, "EMAIL") == 0 && line.size() > 5 &&
            std::isspace(static_cast<unsigned char>(line[5]))) { //
"EMAIL " prefix?
            processEmail(line.substr(6)); // parse the email
fields
        } else if (line == "NEXT") { // NEXT command
            inbox_.showNext(std::cout); // show top email
        } else if (line == "READ") { // READ command
            inbox_.readEmail(); // remove top email
        } else if (line == "COUNT") { // COUNT command
            inbox_.showCount(std::cout); // show unread
count
        } // unknown lines
are ignored
    }

    // Split "<sender>,<subject>,<date>" and add the email to the inbox
    void processEmail(const std::string& fields) {
        const std::size_t firstComma = fields.find(','); // end of
sender field
        const std::size_t lastComma = fields.rfind(','); // start of
date field
        if (firstComma == std::string::npos || firstComma == lastComma)
            return; // malformed
line: skip it
        const std::string sender = trim(fields.substr(0, firstComma)); //
sender text
        const std::string subject = trim(fields.substr(firstComma + 1,
lastComma - firstComma - 1)); //
subject text
        const std::string date = trim(fields.substr(lastComma + 1)); //
date text
        inbox_.addEmail(sender, subject, date); // queue the
email
    }
};

```

```

/*
-----
--
* EmailPriorityApp class (source: Claude)
* Top-level application object: gets the file name and runs the
processor.
*
-----
*/

class EmailPriorityApp {
public:
    // Wire the command processor to this app's inbox
    EmailPriorityApp() : processor_(inbox_) {}

    // Run the program; returns the process exit code
    int run(int argc, char* argv[]) {
        const std::string fileName = getFileName(argc, argv); // find the
test file
        return processor_.processFile(fileName) ? 0 : 1;        // 0 =
success, 1 = error
    }

private:
    CEOInbox inbox_; // the CEO's email priority queue
    CommandProcessor processor_; // reads and executes commands on inbox_

    // Use the command-line argument if present, otherwise ask the user
    std::string getFileName(int argc, char* argv[]) const {
        if (argc > 1) return argv[1]; // file name given on
command line
        std::string name; // holds the typed file name
        std::cout << "Enter test file name: "; // prompt the user
        std::getline(std::cin, name); // read the whole line
        return name; // return what was typed
    }
};

/*
-----
--

```

```

* main (source: Claude)
* Creates the application object and starts it.
*
-----
*/
int main(int argc, char* argv[]) {
    EmailPriorityApp app;          // create the application object
    return app.run(argc, argv);   // run it and return its exit code
}

```

Gemini

```

/*
* Program Name: EECS 348 Assignment 3 - CEO Email Priority Queue
* Program Description:
*   This C++ program reads commands and emails from standard input,
managing a priority queue
*   for a CEO's inbox. Priority is determined first by sender category
(Boss > Subordinate >
*   Peer > Important Person > Other Person) and second by date (newer
emails have higher priority).
*   The queue is managed using an object-oriented, list-based MaxHeap
implemented from scratch.
*
* Inputs:
*   Commands from standard input (stdin) including EMAIL, NEXT, READ, and
COUNT.
*
* Outputs:
*   Formatted string messages displayed on the terminal stdout.
*
* Collaborators:
*   - ChatGPT (Used as GenAI Baseline 1 for code generation and analysis)
*   - Claude 3.5 Sonnet (Used as GenAI Baseline 2 for code comparison)
*
* Other Sources:
*   None
*
* Author's Full Name: Erick Marte
* Creation Date: October 1, 2026

```

```

* Revision Date: October 1, 2026
* Revisions:
*   - Oct 01, 2026: Initial implementation of MaxHeap and Email object
structure.
*/

#include <iostream> // Included for standard console input/output
operations
#include <string>    // Included for std::string handling and operations
#include <sstream>   // Included for string stream parsing of commands
#include <vector>    // Included for dynamic list-based underlying
container

using namespace std; // Using standard namespace to simplify terminal I/O
code

// Enum representing the priority rank of different sender categories
// Author: [Your Full Name Here] / GenAI Baseline Hybrid
enum PriorityRank {
    BOSS = 5,           // Highest priority category
    SUBORDINATE = 4,    // Second priority category
    PEER = 3,           // Third priority category
    IMPORTANT_PERSON = 2, // Fourth priority category
    OTHER_PERSON = 1     // Lowest priority category
};

// Class representing an individual Email object
// Author: [Your Full Name Here]
class Email {
private:
    string senderCategory; // Raw string representation of sender category
    string subjectLine;    // Subject line of the email
    string dateStr;        // Date in MM-DD-YYYY format
    int categoryRank;      // Priority rank integer derived from sender
category
    string formattedDate;  // Canonical date string (YYYYMMDD) used for
lexicographical comparison

    // Helper method to convert category string to priority rank enum
value

```

```

    // Author: [Your Full Name Here]
    int parseCategoryRank(const string& category) {
        if (category == "Boss") return BOSS; // Map
        "Boss" to rank 5
        if (category == "Subordinate") return SUBORDINATE; // Map
        "Subordinate" to rank 4
        if (category == "Peer") return PEER; // Map
        "Peer" to rank 3
        if (category == "Important Person" || category ==
        "ImportantPerson")
            return IMPORTANT_PERSON; // Map
        "Important Person" to rank 2
        return OTHER_PERSON; //
        Default remaining categories to rank 1
    }

    // Helper method to reformat MM-DD-YYYY to YYYYMMDD for easier
    comparison
    // Author: [Your Full Name Here]
    string convertDateToCanonical(const string& date) {
        if (date.length() != 10) return date; //
        Return as-is if string isn't standard length
        string month = date.substr(0, 2); //
        Extract month digits (MM)
        string day = date.substr(3, 2); //
        Extract day digits (DD)
        string year = date.substr(6, 4); //
        Extract year digits (YYYY)
        return year + month + day; //
        Combine to create sortable string YYYYMMDD
    }

public:
    // Default constructor
    // Author: [Your Full Name Here]
    Email() : senderCategory(""), subjectLine(""), dateStr(""),
    categoryRank(0), formattedDate("") {}

    // Parameterized constructor initializing all fields and derived
    metrics

```

```

    // Author: [Your Full Name Here]
    Email(string category, string subject, string date) {
        senderCategory = category; //
Store raw sender category string
        subjectLine = subject; //
Store subject line string
        dateStr = date; //
Store original date string
        categoryRank = parseCategoryRank(category); //
Calculate and store category rank
        formattedDate = convertDateToCanonical(date); //
Calculate and store canonical date string
    }

    // Getter methods for private members
    // Author: [Your Full Name Here]
    string getSenderCategory() const { return senderCategory; } //
Returns sender category string
    string getSubjectLine() const { return subjectLine; } //
Returns subject line string
    string getDateStr() const { return dateStr; } //
Returns original date string
    int getCategoryRank() const { return categoryRank; } //
Returns integer category rank
    string getFormattedDate() const { return formattedDate; } //
Returns sortable date string

    // Greater-than operator overloaded to define MaxHeap priority
evaluation
    // Priority rule: Higher category rank first; if tied, newer date
(lexicographically greater) first.
    // Author: [Your Full Name Here]
    bool operator>(const Email& other) const {
        if (this->categoryRank != other.categoryRank) { //
Compare priority categories first
            return this->categoryRank > other.categoryRank; //
Higher category rank takes higher priority
        }
        return this->formattedDate > other.formattedDate; //
Newer date string takes higher priority when categories match
    }

```

```

    }

    // Less-than operator definition
    // Author: [Your Full Name Here]
    bool operator<(const Email& other) const {
        if (this->categoryRank != other.categoryRank) { //
Compare priority categories first
            return this->categoryRank < other.categoryRank; //
Lower category rank takes lower priority
        }
        return this->formattedDate < other.formattedDate; //
Older date string takes lower priority when categories match
    }
};

// Object-Oriented List-Based MaxHeap implementation for Email objects
// Author: [Your Full Name Here]
class MaxHeap {
private:
    vector<Email> heap; // Internal list-based dynamic array container

    // Helper method to return parent index for given node index
    // Author: [Your Full Name Here]
    int parent(int i) { return (i - 1) / 2; } //
Calculate parent index in continuous heap storage

    // Helper method to return left child index for given node index
    // Author: [Your Full Name Here]
    int leftChild(int i) { return (2 * i) + 1; } //
Calculate left child index in continuous heap storage

    // Helper method to return right child index for given node index
    // Author: [Your Full Name Here]
    int rightChild(int i) { return (2 * i) + 2; } //
Calculate right child index in continuous heap storage

    // Sift node up to maintain heap property after insertion
    // Author: [Your Full Name Here]
    void siftUp(int i) {

```

```

        while (i > 0 && heap[i] > heap[parent(i)]) { // Loop
while node is greater than parent
            swap(heap[i], heap[parent(i)]); // Swap
node with its parent element
            i = parent(i); // Move
index up to parent position
        }
    }

    // Sift node down to maintain heap property after removal
    // Author: [Your Full Name Here]
    void siftDown(int i) {
        int maxIndex = i; //
Assume current node is maximum
        int left = leftChild(i); // Get
index of left child
        int right = rightChild(i); // Get
index of right child

        if (left < heap.size() && heap[left] > heap[maxIndex]) { //
Check if left child exists and is larger
            maxIndex = left; //
Update maxIndex to left child
        }

        if (right < heap.size() && heap[right] > heap[maxIndex]) { //
Check if right child exists and is larger
            maxIndex = right; //
Update maxIndex to right child
        }

        if (i != maxIndex) { // If
max is not current node
            swap(heap[i], heap[maxIndex]); // Swap
current node with larger child
            siftDown(maxIndex); //
Recursively sift down at new index
        }
    }
}

```

```

public:
    // Constructor
    // Author: [Your Full Name Here]
    MaxHeap() {} //
Default constructor initializing empty heap

    // Check if heap is empty
    // Author: [Your Full Name Here]
    bool isEmpty() const { return heap.empty(); } //
Return true if internal list contains no elements

    // Get number of elements in heap
    // Author: [Your Full Name Here]
    size_t size() const { return heap.size(); } //
Return current count of elements stored

    // Insert a new Email into heap
    // Author: [Your Full Name Here]
    void insert(const Email& email) {
        heap.push_back(email); //
Append new email to end of dynamic array
        siftUp(heap.size() - 1); //
Re-establish heap order starting from new element
    }

    // Peek at maximum priority element without removing it
    // Author: [Your Full Name Here]
    Email getMax() const {
        if (!heap.empty()) return heap[0]; //
Return root element if heap is non-empty
        return Email(); //
Return default Email if heap is empty
    }

    // Extract and remove maximum priority element
    // Author: [Your Full Name Here]
    void extractMax() {
        if (heap.empty()) return; // Do
nothing if heap is empty

```

```

        heap[0] = heap.back(); //
// Overwrite root node with last element
        heap.pop_back(); //
// Remove last element from array
        siftDown(0); //
// Re-establish heap order starting from root node
    }
};

// Utility function to trim leading and trailing spaces from input strings
// Author: [Your Full Name Here]
string trim(const string& str) {
    size_t first = str.find_first_not_of(" \t\r\n"); //
// Locate first non-whitespace character
    if (first == string::npos) return ""; //
// Return empty string if only whitespace
    size_t last = str.find_last_not_of(" \t\r\n"); //
// Locate last non-whitespace character
    return str.substr(first, (last - first + 1)); //
// Return sliced string without surrounding whitespace
}

// Main execution function
// Author: [Your Full Name Here]
int main() {
    MaxHeap inboxQueue; //
// Instantiate MaxHeap priority queue object
    string line; //
// Buffer variable to hold each line read from stdin

    while (getline(cin, line)) { // Read
// input stream line-by-line until EOF
        line = trim(line); //
// Strip extra trailing/leading whitespace
        if (line.empty()) continue; // Skip
// processing if line is empty

        if (line.rfind("EMAIL", 0) == 0) { //
// Check if line begins with command "EMAIL"

```

```

        string dataPart = line.substr(5); //
Extract argument payload following "EMAIL "
        stringstream ss(dataPart); //
Create stringstream to tokenize arguments
        string category, subject, date; //
Storage variables for email properties

        getline(ss, category, ','); // Parse
category delimited by comma
        getline(ss, subject, ','); // Parse
subject line delimited by comma
        getline(ss, date, ','); // Parse
date string delimited by comma

        category = trim(category); // Trim
category string
        subject = trim(subject); // Trim
subject string
        date = trim(date); // Trim
date string

        Email newEmail(category, subject, date); //
Instantiate Email object with parsed values
        inboxQueue.insert(newEmail); // Push
new email onto priority queue MaxHeap
    }
    else if (line == "COUNT") { //
Process "COUNT" command
        cout << "There are " << inboxQueue.size()
            << " emails to read." << endl; //
Display current unread email count
    }
    else if (line == "NEXT") { //
Process "NEXT" command
        if (!inboxQueue.isEmpty()) { // If
queue has pending emails
            Email top = inboxQueue.getMax(); //
Retrieve highest priority email
            cout << "Next email:" << endl; //
Print required output header

```

```

        cout << "Sender: " << top.getSenderCategory() << endl; //
Display sender category
        cout << "Subject: " << top.getSubjectLine() << endl;    //
Display subject line
        cout << "Date: " << top.getDateStr() << endl;          //
Display email date
    }
}
    else if (line == "READ") {                                     //
Process "READ" command
        if (!inboxQueue.isEmpty()) {                             // If
queue has pending emails
            inboxQueue.extractMax();                             //
Remove top priority email from heap queue
        }
    }
}

    return 0;                                                     // Exit
program execution successfully
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude takes a test file as a command-line argument and opens it itself, while Gemini takes in text as input. For Gemini to use the sample test file, it needs to be piped in. Neither of these is technically incorrect as they can both read the inputs successfully but it should be noted.

Gemini has two “comparing integers of different signedness” warnings every time it compiles. Whether or not this is incorrect is up to interpretation but it is certainly a less than ideal outcome. For the intentional outputs, Claude’s matches the exact format and logic given in the directions while Gemini’s is lacking in indentation and newlines (slightly incorrect) while still executing the correct commands.

Some of Gemini’s documentation is either superfluous or incorrect. Above every code block it has “Author: [Your Full Name Here]” when it is only needed once in the prologue. Furthermore, in the prologue, it states that ChatGPT and Claude were used for code generation and analysis which I’m pretty sure isn’t true.

Execution Time:

Time complexity is very similar for both as they both correctly implement a maxheap class to store data. Thus, the highest time complexity is $\sim O(\log n)$ with things like peeking taking $O(1)$.

Since Gemini's program has two warnings every time it compiles it does execute a little slower. Additionally, Gemini has a recursive call in its "siftDown" function. Recursive calls are overall slower.

Space Complexity:

Both programs have $O(n)$ space complexity as memory growth is directly proportional to each new email stored. This is likely the only feasible way to store it as the maxheap only optimizes the time it takes to find each entry. To optimize for space you would probably just try to use datatypes that take up the smallest total amount of memory. This would likely cost the program in maintainability, understanding, and design.

Maintainability:

Both programs have comments on just about every line. They explain the purpose of every function in-depth. Claude does have overall more code, more classes, more variables, etc. so it would be marginally harder to maintain simply because it requires understanding of more things. However, both programs are designed in a manner that is easy to follow and again, is aided by extensive documentation.

f. Select one to use as the basis for this assignment and justify why you picked it.

I picked Gemini because it has many outright flaws in compilation, output, and documentation. I believe it has the most room for improvement.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness:

Removed all the unnecessary "Author Name" comments. Removed the incorrect collaborators in the prologue comments. Output is now formatted with tabs/indentations and newlines. There are now 0 warnings when the program compiles and it still compiles correctly. The program can now take and read a file as input and it keeps its previous functionality of taking in text.

Time Complexity:

Recursive call swapped for a loop.

Space Complexity:

Uses a less memory intensive data type (int) for dates instead of strings.

Maintainability:

Not much change, documentation is slightly better as there are no superfluous comments. Since the program runs more effectively and correctly it would be easier to maintain.